Problem 1

Write a Verilog module Vredge for a state machine with one input X and one Moore-type output EDGE, which detects transitions on X. the machine tests its input X at each clock tick and asserts EDGE if the value of X at that tick is different from its value at the previous tick. Use state names A, B, C, and so on as needed. Simulate the FSM to demonstrate functionality. ALWAYS SUBMIT ALL VERILOG CODES.

Answer:

The `Vredge` module is implemented as a Moore Finite State Machine (FSM) designed to detect both rising and falling transitions on input `X`. The design uses four states: `A` and `C` represent stable low and high values, while `B` and `D` represent transition events. The registered output `EDGE` is derived from the next-state logic, ensuring it asserts high within the same clock cycle that a transition is detected. The full Verilog implementation is shown in Figure 2 and Figure 3

The simulation results are presented in Figure 1. As observed in the waveform, the `EDGE` signal asserts high for exactly one clock cycle immediately when a transition on `X` is sampled, confirming the correct functionality of the edge detector.

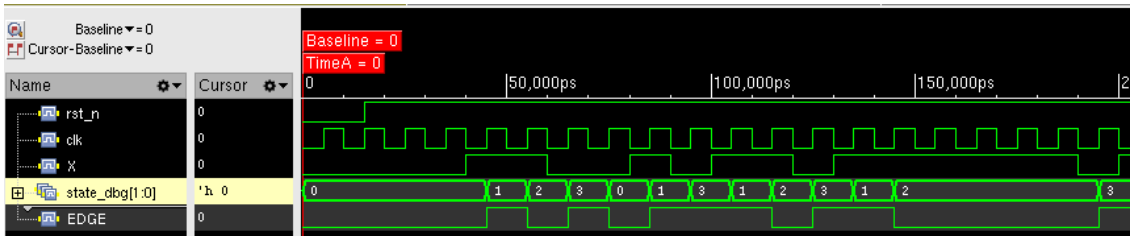

Figure 1: Simulation waveform of the Vredge FSM. The `EDGE` output pulses high correctly upon detecting transitions on input `X`.

```
3   module Vredge (clk, rst_n, X, EDGE, state_dbg);
4       input   clk;
5       input   rst_n;
6       input   X;
7       output reg  EDGE;
8       output [1:0] state_dbg;
9
10      localparam [1:0] A = 2'b00, // stable-0
11                       B = 2'b01, // 0->1 edge
12                       C = 2'b10, // stable-1
13                       D = 2'b11; // 1->0 edge
14
15      reg [1:0] state, next;
16
17      // State register
18      always @(posedge clk or negedge rst_n) begin
19          if (!rst_n)
20              state <= A;
21          else
22              state <= next;
23      end
```

(a) Vredge verilog code part 1

```
25      // Next-state
26      always @* begin
27          case (state)
28              A: next = (X ? B : A);
29              B: next = (X ? C : D);
30              C: next = (X ? C : D);
31              D: next = (X ? B : A);
32              default: next = A;
33          endcase
34      end
35
36      // Registered output
37      always @(posedge clk or negedge rst_n) begin
38          if (!rst_n)
39              EDGE <= 1'b0;
40          else
41              EDGE <= (next == B) || (next == D);
42      end
43
44      assign state_dbg = state;
45  endmodule
```

(b) Vredge verilog code part 2

Figure 2: Verilog code for the edge detector Moore machine

```
`timescale 1ns/1ps

module tb_Vredge;
    reg clk  = 0;
    reg rst_n = 0;
    reg X = 0;
    wire EDGE;
    wire [1:0] state_dbg;

    Vredge dut (
        .clk(clk),
        .rst_n(rst_n),
        .X(X),
        .EDGE(EDGE),
        .state_dbg(state_dbg)
    );

    // 10-ns clock
    always #5 clk = ~clk;
```

(a) Vredge verilog testbench code part 1

```
initial begin
    repeat (2) @(posedge clk);
    rst_n <= 1'b1;

    repeat (2) @(posedge clk);
    X <= 0;

    @(negedge clk); X <= 1;   // 0->1 edge
    @(negedge clk); X <= 1;   // hold
    @(negedge clk); X <= 0;   // 1->0 edge
    @(negedge clk); X <= 0;   // hold
    @(negedge clk); X <= 1;   // 0->1 edge
    @(negedge clk); X <= 0;   // 1->0 edge (back-to-back)
    @(negedge clk); X <= 1;   // 0->1 edge (back-to-back)
    @(negedge clk); X <= 1;   // hold

    // Randomized stress test
    repeat (10) begin
        @(negedge clk);
        X <= $random;
    end

    repeat (3) @(posedge clk);
    $finish;
end
```

(b) Vredge verilog testbench code part 2

Figure 3: Verilog testbench code for the edge detector Moore machine

Problem 2

Create an alternate implementation for the FSM from Problem 1 (Vredge module) using ONLY a ROM block and flip flops. Use the state encoding shown in the table below:

1. A : 00
2. B : 01
3. C : 10
4. D : 11

The general architecture is shown in Figure 4

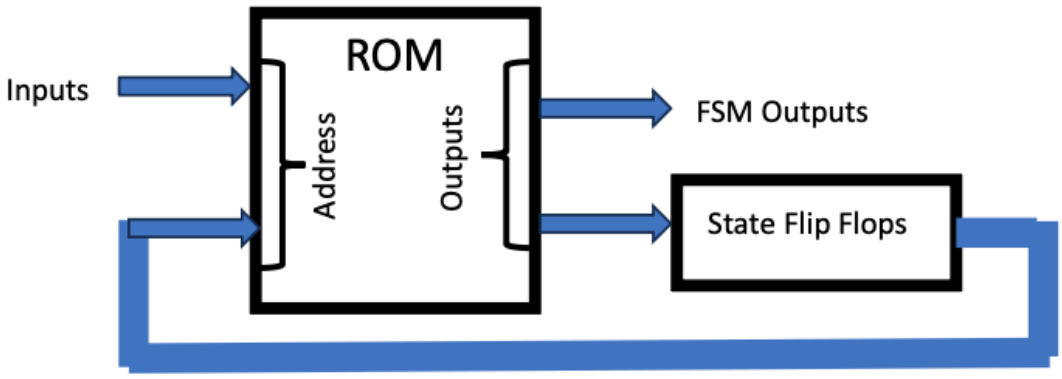

Figure 4: problem 2 scheme

a

What is the size and word width configuration of the ROM for this implementation?

Answer:

The ROM implementation requires an address width based on the inputs to the combinational logic block and a word width based on the outputs.

- **Address Inputs:** The ROM address is formed by the current state (2 bits) and the input signal X (1 bit), totaling **3 address bits**. This results in a ROM depth of $2^3 = 8$ words.
- **Word Width:** The ROM stores the next state (2 bits) and the Moore output *EDGE* (1 bit), requiring a word width of **3 bits**.

ROM Configuration: 8 × 3 (8 words × 3 bits).

b

Show the content of the ROM in a table formatted as shown below

Answer:

The content of the ROM is derived from the state transition table of the Moore FSM. The address consists of the current state bits (Q_1Q_0) followed by the input X, and the output data consists of the next state bits ($Q_1^+Q_0^+$) followed by the output *EDGE*.

ROM Address (State Q_1Q_0, Input X)	**ROM Outputs** (Next State $Q_1^+Q_0^+$, Output *EDGE*)
000	000
001	010
010	111
011	101
100	110
101	100
110	001
111	011

Table 1: ROM Content for Vredge FSM

c

Draw the complete block diagram with all components and connections labeled with the signal names of the Vredge module.

Answer:

The system is implemented using a memory-based state machine architecture consisting of an 8×3 ROM and a 2-bit state register. The complete block diagram is shown in Figure 5.

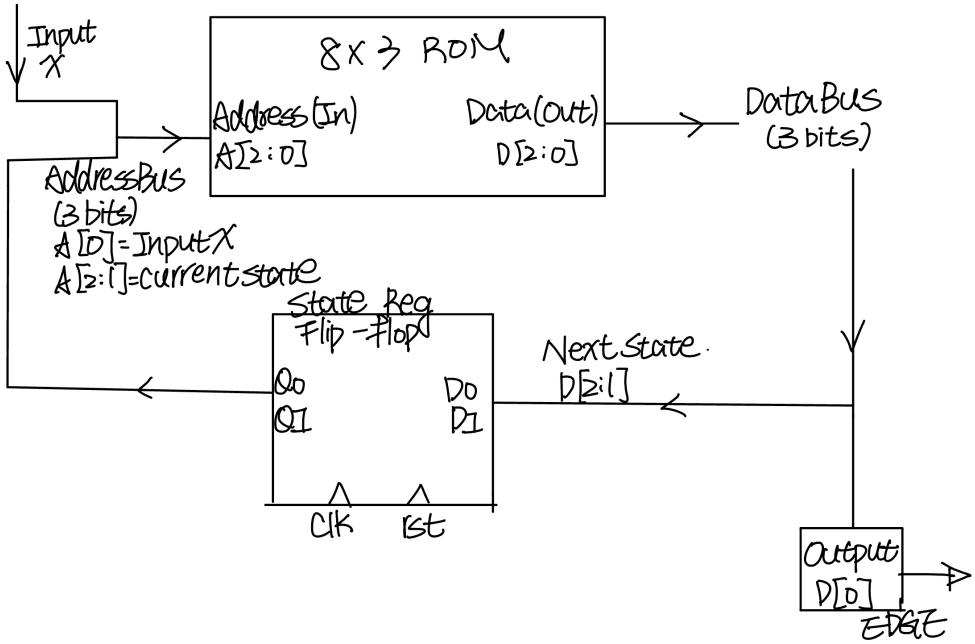

Figure 5: problem 2 block diagram using ROM and state flip flop

The following is a description of the signal names shown in the block diagram for reference.

- **Inputs:**
  - `X`: The external input signal, which drives the least significant bit (LSB) of the ROM address (A_0).
  - `clk`: The system clock that triggers the state flip-flops on the rising edge.
  - `rst_n`: An asynchronous active-low reset signal that initializes the state register to `00` (State A).
- **Internal Connections:**
  - **Address Bus:** The 3-bit ROM address is formed by concatenating the current state bits (`state[1:0]`) with the input `X`.
  - **Data Bus:** The 3-bit ROM output provides the next state logic and the output logic simultaneously. The upper two bits ($D_{2:1}$) drive the `next_state` inputs of the flip-flops.
- **Outputs:**
  - `EDGE`: The Moore output signal is taken directly from the LSB of the ROM data output (D_0), representing the detection of a transition.

Problem 3

Reduce the number of states in the following state table and tabulate the reduced state table

Present State	**Next State**		**Output**	
	X=0	X=1	X=0	X=1
A	F	B	0	0
B	D	C	0	0
C	F	E	0	0
D	G	A	1	0
E	D	C	0	0
F	F	B	1	1
G	G	H	0	1
H	G	A	1	0

Table 2: State table

Answer:

Since the output is depends on the present state and the input X, the FSM here is a Mealy Machine. Based on the implication table analysis:

i. $A \equiv C$: Both have outputs 0,0 and go to (F, B/E).

ii. $B \equiv E$: Both have outputs 0,0 and go to (D, C).

iii. $D \equiv H$: Both have outputs 1,0 and go to (G, A).

We replace $C \to A$, $E \to B$, and $H \to D$. The reduced state table is shown in Table 3.

Present State	**Next State**		**Output**	
	X=0	X=1	X=0	X=1
A	F	B	0	0
B	D	A	0	0
D	G	A	1	0
F	F	B	1	1
G	G	D	0	1

Table 3: Reduced State Table

Problem 4

Assume that the propagation delay from clock to output of a D flip-flop is 5 ns and setup time is 5ns. What is the maximum clock frequency for the 4-bit ripple up counter discussed in Lecture 12 Slide 5?

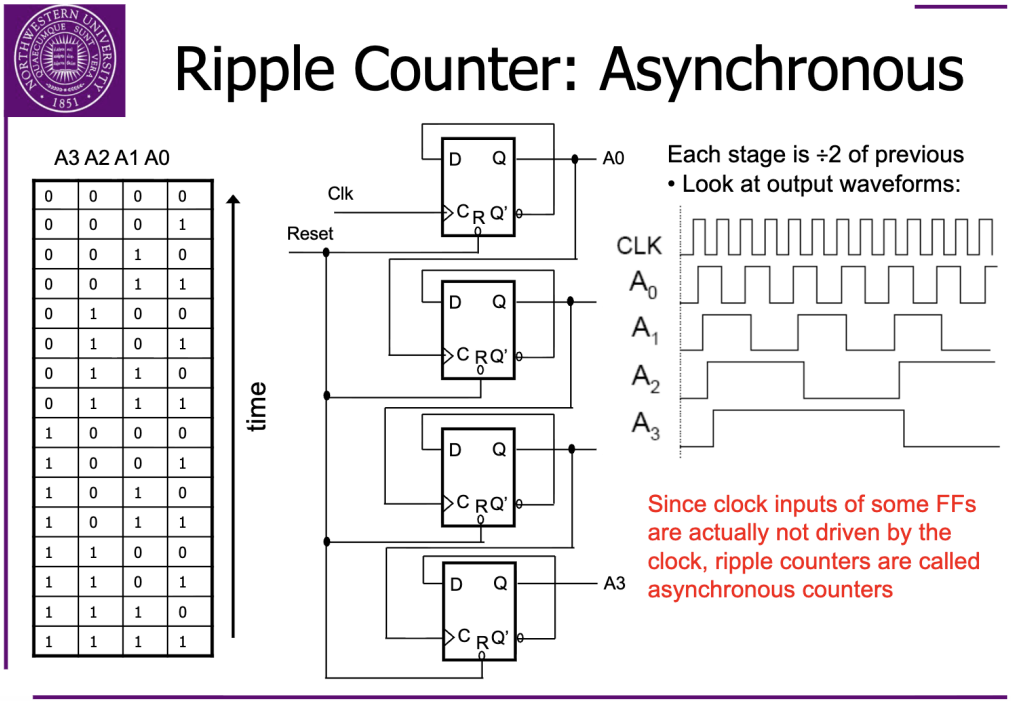

Figure 6: Lecture 12 Slide 5

Answer:

For the 4-bit ripple counter, each flip-flop output change must propagate through all preceding stages. The flip-flops have

$$t_{pcq} = 5 \text{ ns}, \qquad t_{\text{setup}} = 5 \text{ ns}.$$

Since the fourth flip-flop toggles only after all previous stages have propagated, the worst-case total delay from the external clock edge is

$$t_{\text{ripple}} = 4 \cdot t_{pcq} = 4 \cdot 5 \text{ ns} = 20 \text{ ns}.$$

To ensure correct operation, the next clock edge must occur at least one setup time after the most significant bit (MSB) becomes valid:

$$T_{\text{clk}} \geq t_{\text{ripple}} + t_{\text{setup}} = 20 \text{ ns} + 5 \text{ ns} = 25 \text{ ns}.$$

Thus, the maximum clock frequency is

$$f_{\max} = \frac{1}{T_{\text{clk}}} = \frac{1}{25 \text{ ns}} = 40 \text{ MHz}.$$

$$\boxed{f_{\max} = 40 \text{ MHz}}$$

Problem 5

Write a Verilog test bench that instantiates the two versions of a 3-bit counter with decoding in Programs 11-6 and 11-7, and runs them for a few dozen clock ticks. Examine the resulting output waveforms. Describe how they differ.

Answer:

We analyzes two different implementations of a 3-bit counter with an 8-bit active-low decoder output. All output is generated, which one uses combinational logic (Program 11-6) while the other uses registered outputs (Program 11-7).

Program 11-6: 3-bit counter verilog code in Combinational decoder implementation

```
module Vr3bitctrdec ( CLK, CLR, S_L );
    input CLK, CLR;
    output reg [0:7] S_L;
    reg [2:0] Q;
    integer i;
    always @ (posedge CLK) // Create the counter f-f behavior
        if (CLR) Q <= 3'd0;
        else Q <= Q + 1;
    always @ (Q) begin // Decode counter states to create outputs
        S_L = 8'b11111111;
        for (i=0; i<=7; i=i+1)
        if (i == Q) S_L[i] = 0;
    end
endmodule
```

The counter register `Q` is updated on the positive edge of `CLK` using nonblocking assignment (`<=`), while the decoder uses a separate `always @(Q)` block with blocking assignments (`=`). This makes the decoder combinational logic that reacts immediately to changes in `Q`, causing `S_L` to update as soon as `Q` changes without waiting for the next clock edge.

Program 11-7: 3-bit counter verilog code in Registered decoder implementation

```
module Vr3bitctrdec ( CLK, CLR, S_L );
    input CLK, CLR;
    output reg [0:7] S_L;
    reg [2:0] Q;
    integer i;
    always @ (posedge CLK) begin
        if (CLR) Q <= 3'd0; // Create the counter f-f behavior
        else Q <= Q + 1;
        S_L <= 8'b11111111; // Default for outputs is negated
        for (i=0; i<=7; i=i+1) // Decode counter states to assert
        if (i == Q) S_L[i] <= 0; // one active-low output
    end
endmodule
```

Both the counter `Q` and decoder output `S_L` are updated in the same clocked `always` block using nonblocking assignments (`<=`). The decoder output is registered, meaning it updates synchronously with

the clock. However, due to Verilog's nonblocking assignment semantics, `S_L` reflects the value of `Q` from the previous clock cycle rather than the current one, creating a one-cycle delay.

The delay occurs due to Verilog's nonblocking assignment semantics, which operate in two phases. During the evaluation phase, all right-hand sides use current register values, so `Q + 1` and `i == Q` both use the current `Q` (e.g., Q=0). In the update phase, all left-hand sides update simultaneously: `Q` becomes 1 while `S_L` reflects Q=0 (the value used during evaluation). This differs from blocking assignments (`=`), which execute immediately, creating the one-cycle delay in the registered version.

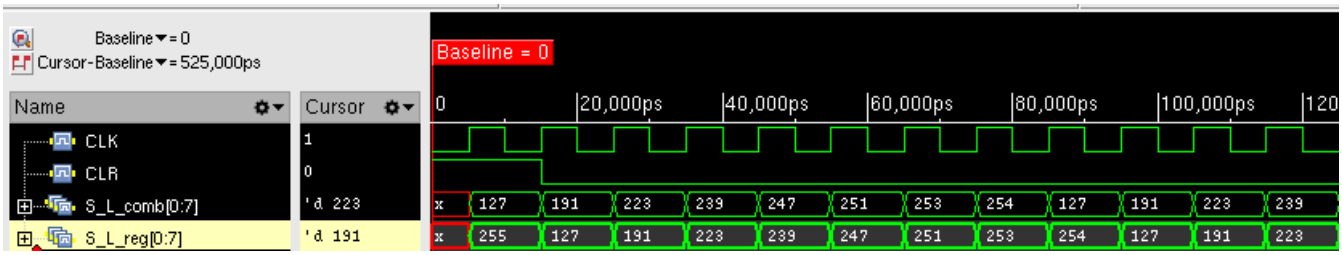

Figure 7: Simulation waveforms comparing combinational (`S_L_comb`) and registered (`S_L_reg`) decoder outputs. `S_L_reg` always reflects the counter state from the previous clock cycle compared to `S_L_comb`.

From the simulation waveforms shown in Figure 7, we observe several key differences. Initially, `S_L_comb` immediately shows the decoded value of the initial counter state (127 = `01111111`, corresponding to Q=7), while `S_L_reg` starts with value 255 (`11111111`), representing the reset state of the output register. During operation, at any given clock edge, `S_L_reg` displays the value that `S_L_comb` held in the previous clock cycle. For example, when `S_L_comb` = 223 (Q=5), `S_L_reg` = 191 (Q=6, which was the previous value of `S_L_comb`). Both implementations correctly decode the 3-bit counter states (0-7) into one-hot active-low outputs, with the sequence 127, 191, 223, 239, 247, 251, 253, 254 representing Q values 7, 6, 5, 4, 3, 2, 1, 0 respectively.

The testbench instantiates both modules with identical inputs (`CLK` and `CLR`) and runs for multiple clock cycles. The simulation confirms that `S_L_comb` updates immediately when `Q` changes, while `S_L_reg` lags by exactly one clock cycle. Both implementations correctly decode all 8 counter states (0-7) and handle reset properly. The combinational decoder (p11_6) provides immediate response but may face timing challenges in high-speed designs, while the registered decoder (p11_7) introduces a one-cycle latency but offers better timing characteristics for pipelined designs.

Problem 6

Revisit the minimized logic function you have identified in Assignment 1, Problem 3.

a

What is the specifications of the smallest PLA that can implement this code scrambler? State the specs in the format of NxMxK, where N is the number of inputs (assume each input is made available in both non-inverted and inverted form), M is the number of AND gates, K is the number of OR gates.
Answer:

From Assignment 1, Problem 3, the minimized sum–of–products form of the function is

$$f(A, B, C, D) = AB + A'B'D + BC'D$$

(one may also choose $A'C'D$ instead of $BC'D$; only three product terms are required).

A PLA needs:

- $N = 4$ inputs: A, B, C, D (each available in true and complemented form),
- $M = 3$ product terms (AND gates) to generate AB, $A'B'D$, and $BC'D$,
- $K = 1$ output (OR gate) to sum these three product terms into f.

Thus, the smallest PLA that can implement this code scrambler has

$$\boxed{N \times M \times K = 4 \times 3 \times 1.}$$

b

Show the connections using a diagram like the one shown below.

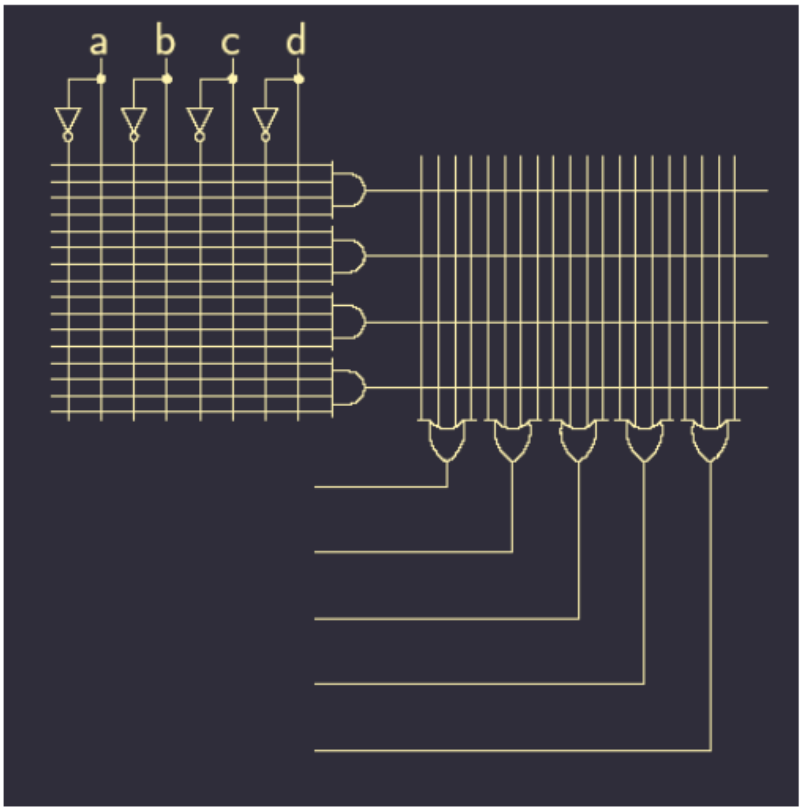

Figure 8: problem 6 scheme

Answer:

The PLA implementation is derived from the minimized sum-of-products expression $f = AB + A'B'D + BC'D$. The connection diagram in Figure 9 illustrates the 4-input, 3-product term, 1-output configuration. The AND plane generates the three required product terms, which are then summed in the OR plane to produce the final output.

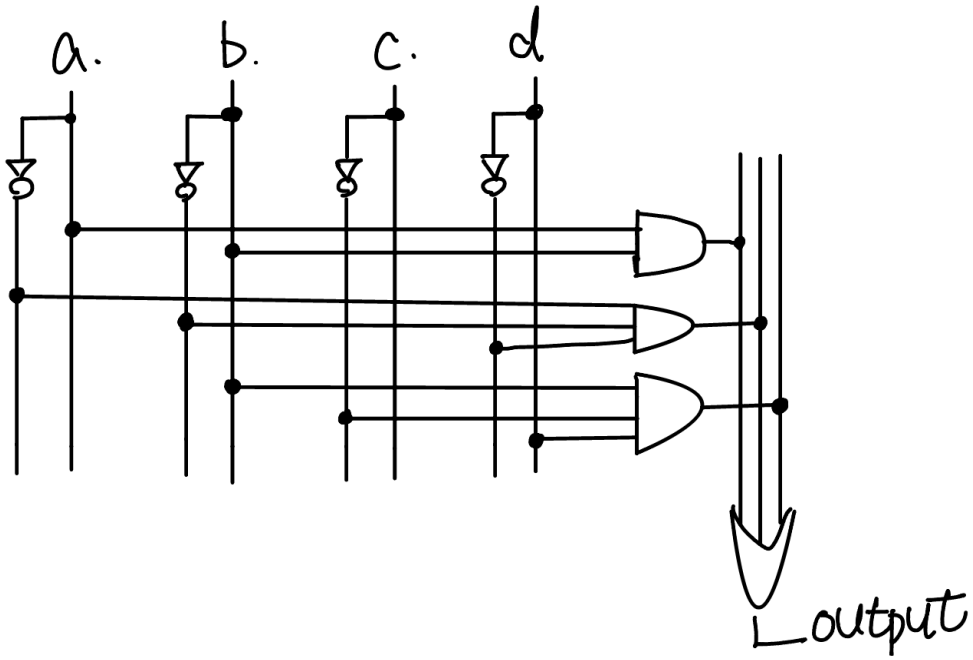

Figure 9: PLA schematic diagram implementing the function $f = AB+A'B'D+BC'D$. The connections represent the programmed fuses in the AND and OR planes.

Problem 7

Revisit Problem 2 of Assignment 5 (Figure 10). Now consider clock skew, which is defined as the constant difference between the time of arrival of the same clock edge at the two consecutive Flip Flops in this circuit. Assume that there is clock skew between FF1 and FF2. The clock arrives at FF1 by t_skew amount of time later than at the FF2. How should the setup time and hold time constraints be re-written factoring in the clock skew? Write the new inequalities by inserting the skew parameter in the correct position.

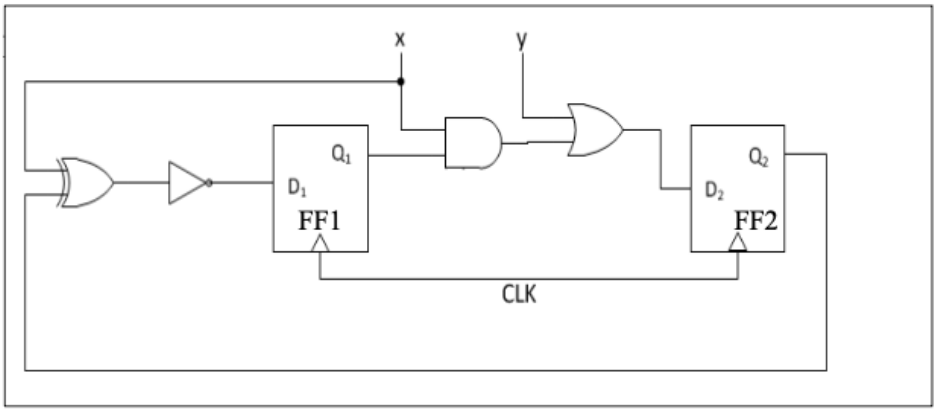

Figure 10: problem 7 scheme

Answer:

Let the clock skew be defined as

$$t_{\text{skew}} = t_{\text{clk,FF1}} - t_{\text{clk,FF2}},$$

and the problem states that the clock arrives at FF1 *later* than at FF2, so $t_{\text{skew}} > 0$. For each sequential path, we rewrite the setup and hold constraints by inserting the appropriate skew term based on the launch and capture flip–flops.

a. **Setup time**

The general setup constraint including clock skew is

$$T_{\text{clk}} \geq t_{pcq} + t_{pd,\max} + t_{\text{setup}} + \big(t_{\text{clk,launch}} - t_{\text{clk,capture}}\big).$$

i. **Path 1:** $Q_1 \to D_2$

FF1 is the launch FF and FF2 is the capture FF. Therefore,

$$t_{\text{clk,launch}} - t_{\text{clk,capture}} = t_{\text{skew}}.$$

The maximum combinational delay along this path is

$$t^{(1)}_{pd,\max} = t_{p,\text{AND,max}} + t_{p,\text{OR,max}} = 10 + 10 = 20 \text{ ps}.$$

Thus, the setup constraint for this path becomes

$$T_{\text{clk}} \geq 10 + 20 + t_{\text{setup}} + t_{\text{skew}} = 30 \text{ ps} + t_{\text{setup}} + t_{\text{skew}}.$$

ii. **Path 2:** $Q_2 \to D_1$

FF2 is the launch FF and FF1 is the capture FF. Thus,

$$t_{\text{clk,launch}} - t_{\text{clk,capture}} = -t_{\text{skew}}.$$

The maximum combinational delay is

$$t^{(2)}_{pd,\max} = t_{p,\text{XOR,max}} + t_{p,\text{INV,max}} = 20 + 5 = 25 \text{ ps}.$$

Hence, the setup constraint is

$$T_{\text{clk}} \geq 10 + 25 + t_{\text{setup}} - t_{\text{skew}} = 35 \text{ ps} + t_{\text{setup}} - t_{\text{skew}}.$$

Since both paths must meet timing, the overall setup constraint is

$$T_{\text{clk}} \geq \max\big(30 \text{ ps} + t_{\text{setup}} + t_{\text{skew}},\ 35 \text{ ps} + t_{\text{setup}} - t_{\text{skew}}\big).$$

For a fixed clock period of $T_{\text{clk}} = 50$ ps, we obtain:

$$t_{\text{setup}} \leq 20 \text{ ps} - t_{\text{skew}} \qquad (Q_1 \to D_2),$$

$$t_{\text{setup}} \leq 15 \text{ ps} + t_{\text{skew}} \qquad (Q_2 \to D_1).$$

Thus, the allowable setup time is determined by the stricter bound:

$$\boxed{t_{\text{setup}} \leq \min\big(20 \text{ ps} - t_{\text{skew}},\ 15 \text{ ps} + t_{\text{skew}}\big)}$$

b. **Hold time**

The general hold-time requirement under skew is

$$t_{pcq} + t_{pd,\text{min}} + (t_{\text{clk,launch}} - t_{\text{clk,capture}}) \geq t_{\text{hold}}.$$

i. **Path 1:** $Q_1 \to D_2$

Again,

$$t_{\text{clk,launch}} - t_{\text{clk,capture}} = t_{\text{skew}}.$$

The minimum combinational delay is

$$t^{(1)}_{pd,\text{min}} = t_{p,\text{AND,min}} + t_{p,\text{OR,min}} = 5 + 5 = 10 \text{ ps}.$$

Therefore:

$$t_{\text{hold}} \leq 10 + 10 + t_{\text{skew}} = 20 \text{ ps} + t_{\text{skew}}.$$

ii. **Path 2:** $Q_2 \to D_1$

Here,

$$t_{\text{clk,launch}} - t_{\text{clk,capture}} = -t_{\text{skew}}.$$

Minimum delay along this path is

$$t^{(2)}_{pd,\text{min}} = t_{p,\text{XOR,min}} + t_{p,\text{INV,min}} = 10 + 2 = 12 \text{ ps}.$$

Thus:

$$t_{\text{hold}} \leq 10 + 12 - t_{\text{skew}} = 22 \text{ ps} - t_{\text{skew}}.$$

Since the design must satisfy both paths, the maximum allowable hold time is

$$\boxed{t_{\text{hold}} \leq \min\big(20 \text{ ps} + t_{\text{skew}},\ 22 \text{ ps} - t_{\text{skew}}\big)}$$